



ABSTRACT

The **AI Codebase Knowledge Builder** transforms any GitHub repository or local codebase into structured, beginner-friendly tutorial documentation — automatically.

Core Problem: Understanding unfamiliar codebases is time-consuming and difficult for newcomers. Manually writing documentation is expensive and rarely kept up-to-date.

Solution: A 6-stage orchestrated LLM pipeline (PocketFlow) crawls source files, identifies core abstractions, analyzes relationships, orders content pedagogically, writes chapters, and assembles a complete Markdown tutorial with Mermaid diagrams.

Observability: Every pipeline stage is instrumented with Prometheus metrics (execution time, memory, CPU, LLM API latency, cache hit rate) and visualized via 6 custom Grafana dashboards.

KEY FEATURES

PocketFlow Orchestration: Lightweight DAG framework with Node/BatchNode pattern, retry logic (max_retries=5, wait=20s), and shared state passing

Google Gemini LLM: gemini-3-flash-preview integration via google.generativeai SDK with JSON prompt/response caching (llm_cache.json)

Flask Web Interface: Full-featured UI with GitHub/local repo input, tutorial viewer with Mermaid diagram rendering, real-time processing feedback

Prometheus Observability: Histograms, Gauges & Counters for every node, function, LLM call and file; scraped every 15s via /metrics endpoint

Multi-Language Output: Tutorials generated in any language; all LLM prompts include language-aware instructions for non-English output

TECHNOLOGY STACK

Backend

- Python 3.10
- Flask 3.x
- PocketFlow (DAG)
- google-generativeai SDK

LLM & Data

- Google Gemini Flash
- YAML prompt parsing
- JSON response cache
- dotenv config

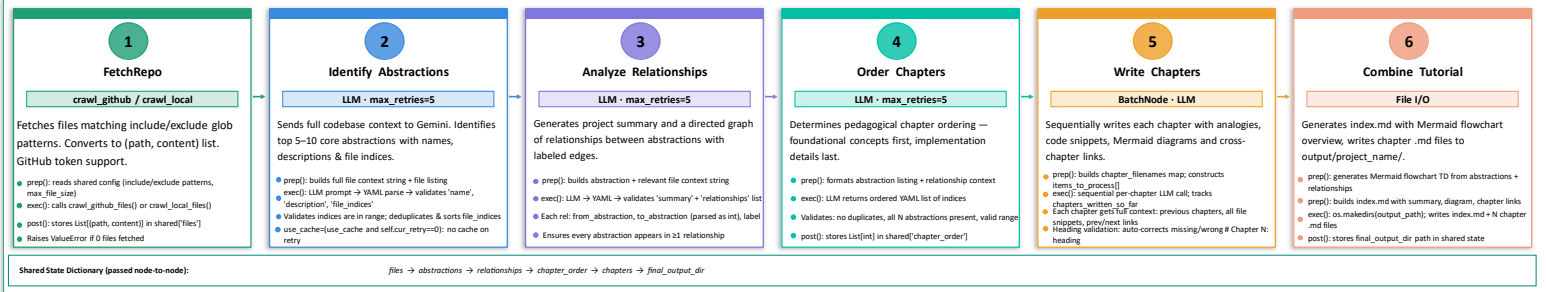
Infrastructure

- Docker Compose
- Prometheus :9090
- Grafana :3000

Monitoring

- Prometheus Client
- Grafana 8+
- psutil (RSS/CPU)
- tracemalloc (heap)

6-STAGE LLM PIPELINE (POCKETFLOW DAG)



FLASK WEB INTERFACE & LLM INTEGRATION

app.py — Flask Routes

- / → renders index.html (form: GitHub URL / local path)
- /api/process → creates tutorial_flow, calls flow.run(shared)
- /tutorial/<name>/<file> → markdown → HTML with Mermaid
- /metrics → Prometheus generate_latest() endpoint

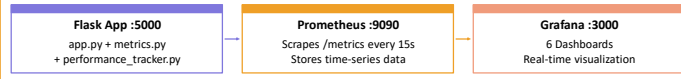
call_llm.py — LLM Gateway

- Configures google.generativeai with GEMINI_API_KEY
- JSON file cache (llm_cache.json): prompt → response
- Cache miss → genai.GenerativeModel.generate_content()
- Records duration/status via MetricsCollector.record_llm_call()

Mermaid Diagram Pipeline

- extract_and_process_mermaid(): regex extracts ``mermaid blocks
- Replaces blocks with <!-- MERMAID_BLOCK_N --> to HTML comments
- Python markdown lib converts remaining Markdown → HTML
- reinject_mermaid_divs(): replaces comments with <div class='mermaid'>

MONITORING STACK ARCHITECTURE



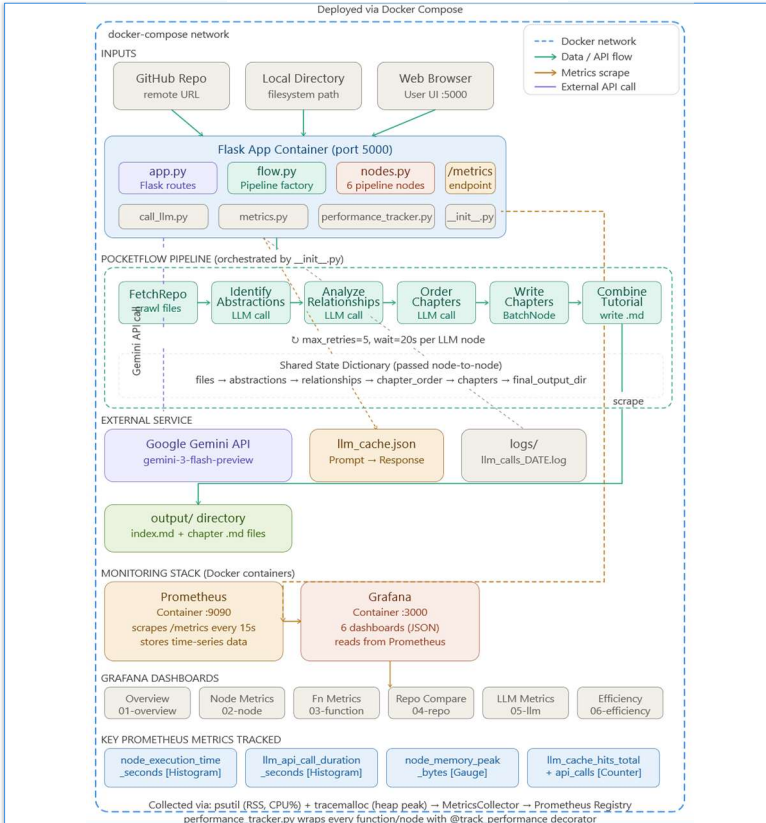
performance_tracker.py — @track_performance Decorator

- Wraps any function: records wall-clock time (perf_counter), peak RSS memory (psutil), heap peak (tracemalloc) and CPU%
- track_execution() context manager for inline code sections without decorators
- On function exit → MetricsCollector.record_function_metrics() updates Prometheus Histograms & Gauges
- Applied automatically to prep(), exec(), post() of each PocketFlow node for full pipeline observability

metrics.py — Prometheus Registry

- Histograms: node_execution_time_seconds, function_execution_time_seconds, llm_api_call_duration_seconds
- Gauges: node/function memory_peak/average_bytes, cpu_percent, total_tutorial_generation_time_seconds
- Counters: llm_api_calls_total (model, status), llm_cache_hits_total, files_processed_total

ARCHITECTURE OF APPLICATION



PIPELINE OUTPUT

index.md

Project summary, Mermaid flowchart of all abstractions & their relationships, numbered chapter listing with links

01_abstraction.md... N_abstraction.md

Per-chapter tutorial with heading, analogies, code snippets (<10 lines), sequence diagrams, inter-chapter links

Mermaid Diagrams

Auto-generated flowchart TD (architecture) and sequenceDiagram per chapter embedded in Markdown

Multi-Language

Full chapter content generated in target language; YAML field names, code syntax remain in English

Browser Viewer

Flask serves rendered HTML with syntax highlighting, copy buttons, Mermaid.js v10 rendering, smooth TOC scrolling

CONCLUSION & FUTURE WORK

Achievements

- End-to-end automated tutorial generation from any codebase in minutes rather than hours
- Full observability: every LLM call, file, and node tracked with sub-second granularity in Grafana
- Robust retry logic (5 retries × 20s wait) handles Gemini API rate limits gracefully
- LLM caching eliminates redundant API calls, reducing cost and latency for repeated runs

Future Directions

- Parallel BatchNode execution: WriteChapters could run chapters concurrently for 6x speedup
- Streaming LLM responses to reduce time-to-first-token and improve UX responsiveness
- RAG (Retrieval-Augmented Generation) for very large repos exceeding LLM context windows
- CI/CD integration: auto-regenerate tutorials on git push via GitHub Actions webhook

GRAFANA DASHBOARDS (6 CUSTOM)

01 Overview

Total generation time (gauge), peak memory (GB), avg CPU%, files processed, abstractions, chapters written, node execution summary table

02 Per-Node Metrics

Execution time over time (time-series), node peak memory MB (stat), node avg CPU% (stat), top 20 nodes by time (table)

03 Function Deep Dive

function_execution_time_seconds (by function_name, node), peak memory per function, avg CPU per function, top 15 slowest functions

04 Repository Comparison

Total time by repository, peak memory by repo (GB), files processed per repo (bar chart) for multi-repo benchmarking

05 LLM Performance

API call rate (5min), cache hit rate (1h), P95 latency (gauge), calls by model (bar), avg duration by model (time-series)

06 Resource Efficiency

Memory per abstraction (MB), time per chapter (s), CPU efficiency trend, top 10 slowest files (time-per-file table)